

FREE PROMPT PACK · CODEX OR CLAUDE CODE

Build Your Own *Jarvis* with GPT-6 Astra

Sixteen copy-paste prompts in two parts. Part One builds the Jarvis you have seen in my videos, thinking with a GPT-6 Astra brain. Part Two adds the new organs: focus sessions that lock your tab and call you out by name, eyes, screen watch, and a brain swap with a personality of its own.

How to use this pack

What you need: a coding agent that can run commands on your machine. Codex running GPT-6 Astra is the natural pairing for this edition; Claude Code works identically. Google Chrome for the microphone and speech. A folder of markdown notes. An OpenAI API key for the brain, and optionally an OpenRouter key if you want to swap brains in Part Two.

Set up in one line. Open a terminal, make an empty folder, start your agent inside it, and set it to GPT-6 Astra. Then paste Prompt 01.

1 **Part Two assumes Part One.**

The organs bolt onto the server and viewer that Prompt 01 creates. Run the foundation first, even if focus sessions are the thing you came for.

2 **Paste each prompt whole, unedited, the first time.**

Every prompt is a complete brief: what to build, the laws it must respect, and how to prove it works. The verification at the end of each one is not optional. It is where the reliability comes from.

3 **Never paste an API key into a chat window.**

Prompt 01 creates a config file with a placeholder. You type the real key into that file yourself. Anything pasted into any chat should be treated as exposed.

4 **Let Astra finish.**

This model is built for long multi-step work: it writes a file, runs it, reads the error, and fixes the cause. Interrupting mid-loop is the most common way to get a worse result. Give it the room.

5 **Direct with feelings, not specifications.**

After the first build, say "it cuts me off," "the callouts repeat," "this feels slow." Describing the problem beats prescribing the fix, because the agent can find the cause and you can only guess at it.

What GPT-6 Astra brings, honestly

Astra shipped on September 3, 2026. It is the brain in this pack for two reasons that matter to a Jarvis, and it is worth being precise about what it is not, because the organs in Part Two are your system, not the model's.

	WHAT IS TRUE	WHAT IT MEANS FOR YOUR BUILD
Long tasks	Built for multi-step work on a computer: it stays oriented, respects task boundaries, and finishes tedious chains.	The prompts in this pack are bigger than earlier editions. One paste builds the galaxy and the brain together. Let it run.
Sight	Reads dense screens, tables and diagrams well.	"What do you think of this?" pointed at your screen becomes genuinely useful. Prompts 06 and 14 lean on it.
Speed	Spends fewer tokens per task than its rivals.	In my build the brain answers a question from my notes in about three seconds , in character. Vision answers in about five.
Cost	\$10 per million input tokens, \$50 per million output.	A whole session of building is a few dollars. Talking to your Jarvis all day is cents. Do not let the headline price scare you off.
Where it is not the leader	Claude Fable 5.1 still leads on deepest reasoning and the coding-agent index, and its cache reads are four times cheaper.	Prompt 08 makes your Jarvis swap brains by voice, so you never have to pick a side. Try both. Keep the one you like.

What is not the model. The focus sessions, the tab lock, the named callouts, the eyes and the desktop card in Part Two are mechanics you build; they work with any brain, and the drift callouts are scripted lines, not Astra thinking. Phone calls and web research in my own Jarvis run on other services. I say this in my videos and you should say it in yours.

Part One — the foundation

Eight prompts that build the Jarvis from the videos: a 3D galaxy of your own notes, a brain that answers from them, a voice both ways, the camera dive to the source, a personality, memory you grow by talking, sight, and a swappable brain with an honesty guard.

If you built the Fable edition already, Prompt 01 is the only one that changed underneath: the brain now calls GPT-6 Astra. Run it in a fresh folder, or tell your agent to update the existing config and the /chat call to the OpenAI API, model gpt-6-astra, and skip to Part Two.

The Galaxy and the Brain

Best for: the first paste. One prompt builds the whole foundation and starts it.

Build me a 3D knowledge galaxy from my markdown notes, and give it a brain I can talk to. Do the whole thing in one pass, then start it and verify it before you tell me it's done.

MY NOTES: [PASTE THE FULL PATH TO YOUR NOTES FOLDER – or delete this line and write "I have no notes, so generate 30 realistic sample notes about a small business into ./notes first"]

THE INDEXER – build.py, Python 3, standard library only. Scan every .md file and write viewer/graph-data.js containing `const GRAPH = {nodes: [...], links: [...]}`. Each node: a label from the filename, its parent folder as a group, roughly a 700-character excerpt. Link two notes when one mentions the other's title or they share `[[wikilinks]]`. CRITICAL: every node gets a numeric id equal to its index in the nodes array. Later prompts look nodes up by index.

THE VIEWER – viewer/index.html, single page, 3d-force-graph from a CDN. No npm, no build step, no framework. Cinematic: black space, a starfield, nodes glowing and colour-coded by group, slow idle rotation. Clicking a node flies the camera to it, lights its neighbours, and opens a side panel with the excerpt.

THE SERVER – server.py, Python standard library, port 4700, serving ONLY the viewer/ folder. Create config.json in the PROJECT ROOT, never inside viewer/, containing `{"openai_api_key": "PUT-YOUR-KEY-HERE", "model": "gpt-6-astra"}`. Do not ask me for the key and never put it anywhere the browser can reach.

THE BRAIN – add POST /chat. Score every note against my question by keyword overlap, weighting title matches higher, take the top 6, and call the OpenAI API with the model from config.json. System prompt: answer ONLY from these notes, in two or three sentences, and say so plainly when the notes do not cover it. Return `{"answer": "...", "nodes": [array indexes used]}`. Keep a short conversation history server-side so follow-ups work. Add a clean input bar at the bottom of the viewer that shows the answer.

FINISH PROPERLY – start the server, confirm the page loads and the graph renders, confirm /chat returns a clean placeholder-key error rather than a crash. Then tell me exactly what to paste into config.json and give me three example questions based on my actual note titles.

Make it yours: if you would rather route the brain through OpenRouter (model `openai/gpt-6-astra`) say so in this prompt, and Prompt 08 will let you swap to Claude, Grok or anything else with the same key.

The Voice, both directions

Best for: the moment it stops being a website and starts being an assistant.

Give it a voice, both directions, using only what the browser already has. No paid speech services.

SPEAKING – every answer is spoken with the Web Speech API. Prefer a British English voice when the system has one. Browsers block audio until the user has interacted with the page, so make the first spoken line fire reliably after my first click, never before.

LISTENING – a microphone button using `webkitSpeechRecognition`. I click it, I speak, the transcript goes through the same /chat flow as typing.

THE PART EVERYONE GETS WRONG – speech recognition finalizes a phrase when I pause, and I pause in the middle of sentences. Do NOT dispatch on the first final result. Buffer it, with the wait in ONE named constant at the top of the file, `FINISH_MS`, set to 900. If more speech arrives before the timer fires, append it and restart the timer; the pause that truly ends my thought sends the whole combined sentence. Short interrupt words like "stop" and "wait" must bypass the buffer and fire instantly.

THE MUTE FLAG – if the page URL contains `?mute=1`, nothing in that tab ever speaks. I will use that for testing so I never hear the assistant from a background tab. Put the check in the one function that speaks, so nothing can route around it.

STATUS – a small status line that shows listening, thinking and speaking, so I always know which of us the machine thinks is talking.

VERIFY – give me a test plan: what to click, what to say, and what I should see and hear, including one deliberate mid-sentence pause to prove the finish window works, and one `?mute=1` tab to prove the silence.

Make it yours: `FINISH_MS` is the single most personal number in this build. Too low and it cuts you off mid-thought; too high and it feels sluggish. Start at 900 and tune it after a day of real use, never in one sitting.

Make it prove where the answer came from

Best for: the ten seconds that make people lean toward the screen.

When it answers from a note, I want the galaxy to prove it.

/chat already returns a nodes array of the indexes it used. Use it:

- Fly the camera to the top source node, light it and its direct neighbours, and open its side panel while the answer is still being spoken, so I can see exactly which note it came from.
- If the answer drew on four or more notes, do not fly anywhere. Light the whole cluster. Flying to one arbitrary node out of six is a lie about where the answer came from.
- Never read the note aloud. It is on screen. The spoken answer stays short.
- Small talk must not drag the camera around. "Good morning" and jokes hold the graph still. Only real questions about my notes move the camera.

That last rule separates a demo from a toy, so implement it deliberately: decide whether the question was about my notes before you decide whether to move the camera.

Then show me: one question that flies to a single note, one that lights a cluster, and one that must not move the camera at all.

Make it yours: this is the shot worth recording. Ask something whose answer lives in one very specific note and film the dive. It reads on video better than any feature list.

The Butler

Best for: turning a question box into something people want to talk to.

Give it a personality. Rewrite the /chat system prompt so the assistant is a dry, impeccably polite British butler with a razor wit. Rules:

- Address me as "sir", occasionally, not in every sentence. Over-using it is the difference between charming and grating.
- Answer in ONE witty sentence plus the facts. Never recite the note back; it is on my screen.
- One genuinely funny line beats three bland ones. If nothing funny is available, be brief instead of forcing it.
- When my notes do not cover something, say so plainly and with a little dignity. Never invent a source, never pad, never pretend a related note is the answer.
- Handle small talk without dragging the camera around the graph.

BOOT GREETING – when the page loads it greets me with the real indexed note count and the correct time of day: "Good evening, sir. 485 notes indexed, all present and accounted for." Read the number from the graph data; never hardcode it.

Put the entire persona in ONE clearly commented block at the top of server.py so I can rewrite the character later without hunting through the file.

Then give me five things to say to it that show off the personality, including one question it definitely cannot answer from my notes.

Make it yours: the butler is just the default. Swap the persona block for a terse mission-control operator, a sarcastic friend, or a patient teacher. Keeping it in one block is what makes that a thirty-second change.

Total Recall

Best for: the point where the brain grows without you opening an editor.

Let me grow the brain by voice.

- When I say or type anything starting with "remember that", POST it to a new /remember endpoint that writes a real markdown file into a captures/ folder inside my notes directory, with a sensible title from the first few words and today's date inside the file.
- Add the new node to the running galaxy LIVE, without a page reload: born at the position of its most related existing node, a brief glow pulse, then fly the camera to it.
- Link it into the graph the same way build.py would, so it is a real citizen of the galaxy and can be a source in the very next answer.
- Confirm out loud in one line, in character.

Two things to get right, because they bite later:

1. Writing a file is not the same as indexing it. The new note must be searchable by /chat in the next question without re-running build.py.
2. Never let a capture silently fail. If the write fails for any reason, say so out loud. A second brain that quietly forgets is worse than no second brain.

Then have me test it: I will say "remember that the finish window should be 900 milliseconds", you show me the new star being born, and I will immediately ask a question that should retrieve it.

Make it yours: add trigger phrases that match how you actually talk: "note to self", "don't let me forget", "log this". The capture only works if the phrase is one you would say without thinking.

The Eyes on your screen

Best for: the first time it answers about something you never described to it.

Give it sight. I want to point at my own screen and ask "what do you think of this?"

- A screen button that starts a `getDisplayMedia` share and holds the stream. While a share is live, show a small, unmistakable indicator. This is not a feature to be subtle about.
- When I ask a question while sharing, grab ONE frame from the stream to a canvas at the moment I ask, encode it as JPEG, and send it to a new POST `/see` endpoint with my question.
- `/see` calls the same GPT-6 Astra brain from `config.json` with the image and the question. Astra reads dense screens well, so tell it to answer specifically about what is on screen and to say plainly when the frame is too small or blurry to judge, rather than guessing.
- The answer comes back through the same voice and on-screen path as every other answer.

BUILD IT SO IT CANNOT LIE TO ME:

- The frame is captured when I ask, never cached from when the share started.
- If the share has ended, it says so. It never answers a screen question from memory of an earlier frame.
- Encode JPEG and send `image/jpeg`. Match the media type to what you actually encode, or the endpoint fails in a way that looks like the whole feature is dead when it is one wrong string.

Verify end to end in front of me: start a share, ask "what am I looking at", and show me the real answer.

Make it yours: screen sharing is far more useful day to day than a webcam and much easier to trust. Point it at a spreadsheet, an error message, or a design and ask what it would change.

The Preflight

Best for: the difference between a weekend demo and something you rely on.

Build me a preflight harness. This is the most important prompt in this pack, so take your time.

Write preflight.py that runs every LIVE chain end to end and prints a verdict. Not unit tests, not mocks. Real calls against the running system, because the failures that hurt are the ones where every unit test passes and a real chain is dead.

Check, in order, with a tick or a cross for each:

- The server is up and serving the viewer.
- The graph data loads and the node count is greater than zero.
- /chat returns a well-formed answer to a real question, with a nodes array.
- The API key in config.json is valid, by one real minimal call.
- The model named in config.json is one the key can actually reach.
- /remember writes a real file and the note is retrievable by /chat immediately afterwards.
- /see answers a real JPEG. Send the media type the client actually sends; a PNG probe can 400 and mimic a dead endpoint.
- The files the browser is actually SERVED match the files on disk. A stale served file is the most common cause of "I fixed it but nothing changed".
- config.json is not reachable from the browser. This one must fail loudly.

Finish with one summary line: N pass, N fail, N warn. Exit non-zero on any failure.

Then run it and show me the output. If anything fails, fix the cause and run it again until it is clean. Do not describe what it would say. Run it.

From now on, whenever you tell me a change is done, run preflight.py first and paste the summary line. "Done" means preflight passed.

Make it yours: add a check every time something breaks on you. A harness that grows one check per real incident becomes the most valuable file in the project within a month. Mine has 43 and every one was earned.

The Brain Swap

Best for: changing which model is thinking, out loud, mid-conversation.

Let me change its brain by voice while it is running.

- Add POST /model that swaps the model at runtime. Spoken names: "switch to Astra", "try on Claude Fable 5.1", "go back to your normal brain".
- Route through OpenRouter using the key in config.json so ONE key reaches any model: "astra" is openai/gpt-6-astra, "fable 5.1" is anthropic/claude-fable-5.1, and so on. Keep the spoken-name map in ONE dictionary near the top of the file. Swaps are runtime only: a restart always returns to the model in config.json, so I can never strand myself on a brain I did not mean to keep.
- Show the current brain on screen as a small chip. Labels: only a hyphen between two digits is a version dot, so "gpt-6-astra" reads GPT 6 ASTRA and "fable-5-1" reads FABLE 5.1.

NOW THE RULE THAT MAKES THIS SAFE, AND I MEAN THIS ONE:

Keep an explicit set of the real model ids you know exist. When I name a family plus a version, build the candidate id and check it against that set. If it is not there, REFUSE and tell me what you do have. Never fall back to the nearest match.

The failure this prevents is specific and nasty: I say "opus 5", a loose matcher sees "opus", throws away the version, loads an older Opus, and cheerfully announces it did what I asked. I then spend an hour testing the wrong model. An honest error is worth more than a helpful guess, every single time.

Test it in front of me: swap to a real model, swap to a version that does not exist and show me the refusal, then restart the server and show me it came back on the config model.

Make it yours: the refusal rule is the transferable lesson. Anywhere your build maps something a human said onto something exact, make the near-miss fail loudly instead of guessing. Silent wrong answers cost more than loud errors.

Part Two — the new organs

Eight prompts for the features I built in the last two weeks: an accountability timer that locks your tab and calls you out by name, the deferred lock that solves the "the button is in the wrong tab" problem, an explicit "lock this tab" from voice and from a desktop button, camera eyes with a relief valve, a screen watch that nudges when you stare, a brain swap with a personality, and the field tooling that made all of it debuggable.

These prompts carry the laws I learned the hard way, written into the brief so your agent respects them from the first build. Each one still ends with a verification. Run them in order; 10 and 11 assume 09.

Focus Sessions

Best for: the feature that turns an assistant into something that keeps you honest.

Build me focus sessions: an accountability timer that watches where I actually am and calls me out when I drift.

START – "thirty minutes on this" by voice, or a FOCUS button in the viewer. The session lives on the SERVER with a one-second tick, so a reloaded tab rejoins it and the countdown never dies.

THE TARGET – lock the frontmost APP by bundle id, and when that app is Chrome-family, also lock the active TAB by hashing the URL's HOST, never the full URL: my work site is a single-page app whose path changes with every click, and site-level is the honest granularity. Read the frontmost app with a FRESH query every tick from a command-line tool; do not use a cached notification API, which freezes in a long-lived headless server and reports the first app forever.

PRIVACY, STRUCTURALLY – identities are compared and discarded inside the reader. The state the client sees is a whitelist of booleans and counters. Write a test that proves no app or tab identity can appear in it.

DRIFT – grace of 800ms for an app or tab switch, then a spoken callout from canned pools that escalate over three tiers; a nag cadence while one drift persists, settable by voice ("call me out every thirty seconds"); a snooze ("give me fifteen seconds"); an excuse ("it's okay, I'm doing research") that refunds the current excursion and stays quiet until I am back; pause, resume, extend and abort by voice. The Jarvis tab itself is HOME BASE: coming back to talk to it is never a drift.

THE END – a spoken report card: minutes on target out of planned, drifts, a streak that grows when a session is at least 85 percent clean, kept in an aggregates-only ledger. A desktop countdown card pinned top-right that tints on drift.

Every knob is a named constant at the top of its file. Verify with a live loop: switch tabs, hear the callout within three seconds, switch back, end the session and hear the report.

Make it yours: tune the knobs from field numbers, never from memory. If a callout feels late, print the tick timings before you touch the grace. Guessing at these cost me six rounds.

The Deferred Lock

Best for: fixing the bug you will hit in your first five minutes.

The FOCUS button lives inside the Jarvis tab. So locking "the active tab" at click time locks the one tab I am guaranteed to leave, and my real work then reads as a drift. Fix that properly.

DEFER – when the session starts from the Jarvis tab, or when the tab read fails at the click moment (the viewer's 3D boot can stall the browser's scripting for a few seconds), do NOT lock anything yet. Wait for me to settle: the first non-Jarvis surface I stay on for two consecutive ticks becomes the app and tab target, and the assistant says "Locked on, sir." so a wrong lock is audible.

ONLY WHERE I AM CAN SETTLE – never guess the target from a background window or a second monitor. With three browser windows open, a guess locks a tab I never looked at, three seconds into the session. A wrong lock is worse than no tab lock: if I never leave the Jarvis tab, fall back to app-only after about 45 seconds instead.

SAY IT – while the lock is deferred the start line must tell me: "Go to what you're working on and I'll lock on there. And what are we focusing on?" A deferred lock nobody mentions is the same as a broken one. Expose the deferred state as a boolean in the session state so the status line and the debug panel can show it.

THE ANSWER WINDOW – after that question, open the microphone for one answer without a wake word, attach my answer to the running session as its intent, and use it to colour the first callout and the report card. Never parrot a long dictated intent back to me; "Noted, sir" is enough.

Verify with the exact click flow: click FOCUS on the Jarvis tab, hear the go-to-your-work line, switch to a work tab, hear "Locked on, sir.", switch away, hear the callout.

Make it yours: make the settle count and the deferred flag visible in a debug overlay behind a URL flag. When someone says "it locked the wrong thing," those two numbers are the whole diagnosis.

Lock This Tab

Best for: changing what is locked without aborting the session.

Give me an explicit way to move the lock, by voice and by a physical button.

BY VOICE – "lock on this tab", "keep me in this tab", "this is the tab", "stay on this tab", and the natural version with a lead-in: "okay, I'm gonna need you to keep me in this tab." From the tab I want: lock it now, silently forgive the current drift, say "Locked on, sir." From the Jarvis tab: re-arm the deferred lock and say "Go to it, sir – I'll lock on where you land." From a non-browser app: lock the app. Place this route ahead of any pixel-share "lock my screen" command so that mid-session "lock this tab" means the target, not the share.

BY BUTTON – a "LOCK THIS TAB" pill on the desktop countdown card, next to the pause and abort controls, that flashes LOCKED for a second and posts the same re-target with a flag saying it came from the card.

THE CARD TRAP – a click on the desktop card can make the card's own process the frontmost app, so a naive re-target would lock the card. When the request came from the card, or whenever the frontmost process is your own, read the browser's FRONT WINDOW directly through its scripting interface instead; it answers even with the browser in the background.

TEACH THE CHAT BRAIN – put the recovery in the assistant's system prompt so it never invents an impossible one: to change the target, go to the tab and say "lock on this tab". Never tell me to bring a tab to the front and then click FOCUS, because the button lives in the Jarvis tab; never tell me to abort and restart just to move the target.

Verify from three surfaces: a work tab, the Jarvis tab, and the desktop button.

Make it yours: the card trap generalizes to every overlay you ever build: a button in a floating window must never trust "frontmost" to mean the user's app. Read the thing you actually care about.

Name the Distraction

Best for: the line everyone quotes back to you: "Sir, Instagram can wait."

When I drift, name what I drifted into. "Sir, Instagram can wait." "Twice into YouTube, sir. It is starting to look deliberate." "Third time in Slack, sir. The detours are becoming the project."

THE LABEL – for a browser drift, derive a spoken name from the host: a map for the big distractions (Instagram, YouTube, X, Reddit, TikTok, Netflix, Gmail) and the bare domain for anything else, so a random site reads "Sir, example.com can wait." For an app drift, use the app's display name from the same tool that reports the frontmost app. The Jarvis tab never gets a label; it is home base.

THE LINES – named pools with a placeholder, FOUR lines per escalation tier so a long drift does not loop three phrases, and a nameless pool as the fallback. The first callout of a session names both the work I told you about and the distraction: "Sir – Instagram does not look like 'the thumbnail sprint' to me." A drill-sergeant pool that names it too, for the days I ask for that.

THE PRIVACY LAW – the label rides ONE engine tick into the spoken line and is stored NOWHERE: not the session state, not the ledger, not the episode log, not the assistant's notes. Write a test that uses a made-up label nobody would ever write in canned text, drives a drift, and proves the label is absent from state, the report and the notes. A test that greps for "Instagram" will be fooled by your own example sentences. A switch at the top of the file turns naming off.

TELL THE TRUTH – update the assistant's privacy line: it names the distraction out loud in the moment and never writes it down.

Verify: drift into a mapped site, an unmapped site, and a desktop app, and read me the three lines.

Make it yours: this is the beat that gets quoted in comments. Write six lines in your own voice for the site that actually gets you, and put the funniest one in tier one, where it will be heard most.

The Eyes

Best for: the assistant that notices you picked up your phone.

Give it eyes. A webcam organ that notices posture, with the camera frames never leaving the machine.

LOCAL ONLY – run face and pose landmarks in the browser. Publish booleans to the session: present, head down (the phone posture), slouched. No frame is uploaded for this; the numbers are the whole signal.

NUDGES – a sustained phone or slouch posture for 700 milliseconds earns one dry spoken nudge, then a 30-second cooldown. Absence gets a longer grace, twelve seconds, because glancing at a notification is not leaving. The eyes feed the focus session so a phone drift is counted like a tab drift, with its own line pool.

LOOK – "look at me" and "what do you think of my shirt?" send ONE frame to the GPT-6 Astra brain with a prompt that says this is a live webcam photo of the user at their desk, answer their question with dry wit in one to three sentences. Do not route "what do you think of this screen" here; that phrase belongs to the screen organ.

THE RELIEF VALVE – "no Jarvis, I need to do something important" or "give me a minute" silences every nudge for three minutes.

THE EAR LAW – no organ may turn the microphone on. If the eyes come on while the mic is off, the assistant must say so: "My ears are off, sir – tap the ear button and just talk." An organ may open the conversation; it may never open the microphone. And every tab I use for testing loads with ?mute=1, so I never hear the assistant from a background tab.

Verify: pick up your phone and get nudged inside a second, then say the relief line and prove the silence.

Make it yours: the sustain times and cooldowns are taste. 700 milliseconds is demo speed; for real work I run slower and quieter. Named constants, tune from a day of use.

Screen Watch

Best for: the organ that notices you have been staring at the same thing for a minute.

Add a screen watch that costs nothing until it needs to think.

"WATCH MY SCREEN" – start a display share of the ENTIRE SCREEN. Every five seconds, shrink a frame to a tiny thumbnail and diff it locally against the last one. Zero cost, zero frames leave the machine. The moment the screen changes, reset a stillness clock.

THE NUDGE – when the screen has sat unchanged past 60 seconds, send ONE frame to the brain and ask for a dry, genuinely useful nudge about what I am stuck on. Speak it. Then a three-minute cooldown.

ANSWER FROM THE SAME SHARE – while watching, "what do you think of this?" and any research question use this share's frame, so I never need two shares open. This organ is screen-only: it never touches the camera. Want both? Turn both on.

THE TAB-SHARE TRAP – a captured single tab can never police tabs: it keeps rendering itself while I wander, and its top strip is page content, so scrolling reads as change. If I pick a tab in the share picker, say so honestly and steer me to share the entire screen. Open the picker on Entire Screen by default.

THE FACE – while watching, a small card with the assistant's face pinned to the top-right corner of the real desktop, over the menu bar, across every Space, so I always know it is watching.

Verify: watch, stare at one screen for the threshold, hear the nudge; then ask about the screen and get an answer from the same share.

Make it yours: the stare threshold is the personality of this organ. Sixty seconds is a demo setting; four minutes is a colleague who respects deep work. Pick the one you would tolerate on a Tuesday.

The Swap Gets Its Lines

Best for: making a brain swap sound like the assistant, not a status message.

When I swap brains, the assistant asks the NEW brain to introduce itself in one dry sentence. Reasoning models often answer that with nothing, so it falls back to "New brain fitted, sir – openai/gpt-6-astra online", which is the least funny sentence in the project. Fix it.

CURATED LINES – a list of (pattern, lines) pairs near the top of server.py. When the new model id matches a pattern, use a scripted line instead of asking the model. Give GPT-6 Astra a pool of four to six lines in the butler's voice that ROTATE per swap, so repeated swaps never repeat. Write them about the things I actually do at this desk, not about the model's benchmarks. Two to set the tone, then write the rest yourself:

"New brain fitted, sir – GPT-6 Astra. Do try to keep up."

"GPT-6 Astra online, sir. I now understand everything – except why you keep opening Instagram."

ONE VOICE – every door that swaps a brain (the button, the spoken sentence, the control tag the chat brain emits) must go through ONE function that consults the curated list first, then the model's own intro, then a fallback that reads a pretty name ("GPT-6 Astra"), never a raw slug.

PINNED NAMES – put the spoken forms in the config's model aliases (astra, gpt-6 astra, gpt 6 astra, gpt-6) so a catalogue change can never hand me a "-mini" mid-take.

Verify: swap by button, then by voice, and show me two different lines.

Make it yours: write a pool for whichever brain you use most. The line that lands is the one about the thing you actually do at your desk.

Instrument before you fix

Best for: the prompt that saves you from the six silent rounds I lost.

Build the tooling that tells me the truth when a feature "isn't working", so I never guess.

GET /focus/diag – booleans and statuses ONLY, no identities: is the frontmost app readable, is it a browser, the tab read status, is the front tab the Jarvis tab, is a hash present, is the lock deferred, how many settle ticks, is there an app target, a tab target, am I on target right now by each lane, is the session on, is the tick thread alive. Read it from the RUNNING server, because a fresh process can pass every check while the long-lived one is frozen.

?focusdebug=1 – an overlay in the viewer showing the session flags, the deferred state, the last pixel diff and gate, drifts, excused, and whether the intent window is open.

THE LEDGER – the end-of-session record with whitelisted keys only: timestamp, planned minutes, active minutes, on-target minutes, drifts, seconds adrift, percent, completed. Write a test that proves nothing else can be stored.

THE PROBE PAGE – ?focusprobe=1 runs the hermetic battery in the browser and writes PASS/FAIL per case into the page title, so a script can read the verdict. Run it at an asserted viewport of 1440 by 900; hidden tabs throttle timers and make time-based checks lie.

THE LAW – when a feature fails in the field, read the diag and the ledger, count the browser windows, and check which host the assistant's own tab is on BEFORE touching code. When several fixes change nothing, the input is lying: reproduce my exact flow with the instruments on.

Run the probe and preflight, and show me both summary lines.

Make it yours: every silent failure I ever had was a sensor that lied while the tests passed. Build the truth endpoints first and the features get easier, not harder.

Five things I learned putting GPT-6 Astra inside Jarvis

1 **The brain is the easy part now.**

Astra answers from my notes in three seconds and reads my screen in five. Everything that makes it feel alive, the eyes, the lock, the callouts, the report card, is mechanics you build once and keep across every brain that ships after this one.

2 **Longer prompts beat more prompts.**

Part One's foundation is one paste where it used to be three. Astra holds the whole build in its head; every handoff you add is a chance to lose context it was perfectly capable of keeping.

3 **A wrong lock is worse than no lock.**

Never let the system guess where you are from something you are not looking at. Defer, settle, announce. That rule fixed the bug I had lived with for a week.

4 **Name things out loud, store nothing.**

"Sir, Instagram can wait" is the line people repeat, and it costs no privacy if the name lives one tick and dies. Prove it with a test that uses a word that appears nowhere else.

5 **Build the truth endpoints first.**

Every hour I lost on this project was a sensor lying while the tests passed. Diag, probe, preflight. Then features.

Want the full system, not just the prompts?

This pack gets you a working second brain with organs. Inside the paid AI Workshop community you get the complete builds I actually run: the phone line, the desktop card, the automation registry, and the skill installs behind all of it, plus the playbooks for turning these into paid client work.

Free community: skool.com/aiworkshop-lite

Full workshop: skool.com/aiworkshop

YouTube: [Zubair Trabzada | AI Workshop](#)